

Trabalho: Aplicação Distribuída com RPC

Objetivo

Desenvolver uma aplicação distribuída no paradigma cliente/servidor usando RPC (Remote Procedure Call) para um problema escolhido pelo grupo. A aplicação servidora deve armazenar os dados de forma simples e fornecer uma API em RPC para as funções definidas. O cliente pode ser executado em linha de comando ou ter uma interface gráfica simples, utilizando os métodos fornecidos pela API do servidor.

Abaixo estão algumas sugestões de aplicações:

1. Sistema de Agendamento de Consultas Médicas

API do Servidor

- `agendar_consulta(paciente, medico, data, hora)`
 - **Descrição:** Verifica se o médico está disponível no horário informado e agenda a consulta.
 - **Retorno:** Sucesso ou mensagem de erro ("Horário já ocupado", "Médico não encontrado", etc.).
 - `cancelar_consulta(paciente, medico, data, hora)`
 - **Descrição:** Cancela uma consulta previamente agendada.
 - **Retorno:** Sucesso ou erro ("Consulta não encontrada", "Não é possível cancelar consulta passada", etc.).
 - `consultar_consulta(medico, data, hora)`
 - **Descrição:** Informa se há uma consulta marcada para o médico no horário indicado.
 - **Retorno:** Detalhes da consulta ou "Horário livre".
 - `consultar_agendamentos_medico(medico)`
 - **Descrição:** Lista todas as consultas futuras do médico.
 - **Retorno:** Lista de datas, horários e pacientes.
 - `consultar_agendamentos_paciente(paciente)`
 - **Descrição:** Lista todas as consultas agendadas para o paciente.
 - **Retorno:** Lista com médico, data e hora.
-

2. Sistema de Controle de Estoque

API do Servidor

- `adicionar_item(produto, quantidade)`
 - **Descrição:** Adiciona uma quantidade ao estoque do produto.

- **Retorno:** Confirmação da nova quantidade em estoque ou erro ("Produto inexistente").
 - `remover_item(produto, quantidade)`
 - **Descrição:** Remove quantidade do estoque, caso haja disponível.
 - **Retorno:** Sucesso ou erro ("Estoque insuficiente", "Produto não encontrado").
 - `consultar_estoque(produto)`
 - **Descrição:** Retorna a quantidade atual disponível em estoque.
 - **Retorno:** Número inteiro ou erro.
 - `listar_estoque()`
 - **Descrição:** Retorna a lista de todos os produtos com suas quantidades.
 - **Retorno:** Lista de tuplas (produto, quantidade).
 - `historico_movimentacoes(produto)`
 - **Descrição:** Exibe o histórico de entradas e saídas do produto.
 - **Retorno:** Lista de movimentações com data, tipo (entrada/saída) e quantidade.
-

3. Sistema de Alocação de Equipamentos de TI

API do Servidor

- `alocar Equipamento(usuario, equipamento, data_inicio, data_fim)`
 - **Descrição:** Verifica disponibilidade do equipamento no período e registra alocação.
 - **Retorno:** Sucesso ou erro ("Equipamento já alocado nesse período").
- `devolver_equipamento(usuario, equipamento)`
 - **Descrição:** Registra a devolução do equipamento pelo usuário.
 - **Retorno:** Sucesso ou erro ("Nenhuma alocação ativa encontrada").
- `verificar_disponibilidade(equipamento, data)`
 - **Descrição:** Informa se o equipamento está disponível na data indicada.
 - **Retorno:** Disponível / Alocado + dados da alocação.
- `consultar_alocacoes_usuario(usuario)`
 - **Descrição:** Lista os equipamentos alocados ao usuário.
 - **Retorno:** Lista com equipamentos, períodos e status (ativo/finalizado).
- `consultar_alocacoes_equipamento(equipamento)`
 - **Descrição:** Lista o histórico de alocações do equipamento.
 - **Retorno:** Lista com usuários, datas e status.

4. Sistema de Reservas de Quadras Esportivas

API do Servidor

- `reservar_quadra(usuario, quadra, data, hora)`
 - **Descrição:** Verifica se a quadra está livre e registra a reserva.
 - **Retorno:** Sucesso ou erro ("Quadra já reservada nesse horário").
- `cancelar_reserva(usuario, quadra, data, hora)`
 - **Descrição:** Cancela uma reserva ativa feita pelo usuário.
 - **Retorno:** Sucesso ou erro ("Reserva inexistente" ou "Não autorizada").
- `verificar_reserva(quadra, data, hora)`
 - **Descrição:** Informa se há reserva para a quadra no horário indicado.
 - **Retorno:** Reservada (usuário) ou Disponível.
- `reservas_usuario(usuario)`
 - **Descrição:** Lista todas as reservas futuras feitas pelo usuário.
 - **Retorno:** Lista com quadra, data e horário.
- `reservas_quadra(quadra)`
 - **Descrição:** Lista todas as reservas feitas para uma quadra.
 - **Retorno:** Lista com usuários, datas e horários.

5. Sistema de Empréstimo de Livros em Biblioteca

API do Servidor

- `realizar_emprestimo(usuario, livro, data_emprestimo)`
 - **Descrição:** Verifica se o livro está disponível. Se estiver, registra o empréstimo para o usuário na data informada.
 - **Retorno:** Sucesso ou erro ("Livro indisponível", "Livro não encontrado", "Usuário com limite de empréstimos atingido").
- `devolver_livro(usuario, livro)`
 - **Descrição:** Verifica se o livro está emprestado ao usuário. Se estiver, registra a devolução.
 - **Retorno:** Sucesso ou erro ("Empréstimo não encontrado", "Livro já devolvido").
- `consultar_livro(livro)`
 - **Descrição:** Retorna se o livro está disponível ou emprestado e, se emprestado, para qual usuário e desde quando.

- **Retorno:** Disponível / Emprestado para [usuário] desde [data].
- consultar_emprestimos_usuario(usuario)
 - **Descrição:** Retorna lista de todos os livros atualmente emprestados ao usuário, com data de empréstimo e status (no prazo/atrasado).
 - **Retorno:** Lista com título do livro, data e status ou mensagem ("Nenhum empréstimo encontrado para o usuário").
- consultar_emprestimos_livro(livro)
 - **Descrição:** Retorna histórico de empréstimos do livro, incluindo usuário, data de empréstimo e data de devolução.
 - **Retorno:** Lista com usuário, data_emprestimo, data_devolucao ou mensagem ("Nenhum histórico de empréstimos encontrado para o livro").

6. Sistema de Gerenciamento de Tarefas (To-Do List Compartilhada)

API do Servidor

- adicionar_tarefa(usuario, descricao, prazo, prioridade)
 - **Descrição:** Adiciona uma nova tarefa para o usuário.
 - **Retorno:** ID da tarefa ou erro ("Usuário não encontrado").
- marcar_concluida(usuario, id_tarefa)
 - **Descrição:** Marca uma tarefa como concluída.
 - **Retorno:** Sucesso ou erro ("Tarefa não encontrada" ou "Não autorizado").
- listar_tarefas(usuario, filtro)
 - **Descrição:** Retorna tarefas do usuário (todas, pendentes ou concluídas).
 - **Retorno:** Lista de tarefas com descrição, prazo e status.
- compartilhar_tarefa(usuario_origem, usuario_destino, id_tarefa)
 - **Descrição:** Compartilha uma tarefa com outro usuário.
 - **Retorno:** Sucesso ou erro ("Usuário destino não existe").

7. Sistema de Votação Online (Enquetes)

API do Servidor

- criar_enquete(usuario, pergunta, opcoes, data_limite)
 - **Descrição:** Cria uma nova enquete com opções de resposta.
 - **Retorno:** ID da enquete ou erro ("Usuário não autorizado").
- votar(usuario, id_enquete, opcao)
 - **Descrição:** Registra um voto em uma enquete.
 - **Retorno:** Sucesso ou erro ("Enquete encerrada" ou "Voto já registrado").

- consultar_resultados(id_enquete)
 - **Descrição:** Retorna os votos acumulados por opção.
 - **Retorno:** Lista de opções e contagem de votos.
 - listar_enquetes_ativas()
 - **Descrição:** Retorna enquetes abertas para votação.
 - **Retorno:** Lista com ID, pergunta e data de encerramento.
-

8. Sistema de Monitoramento de Sensores (IoT)

API do Servidor

- registrar_sensor(id_sensor, tipo, localizacao)
 - **Descrição:** Cadastra um novo sensor no sistema.
 - **Retorno:** Sucesso ou erro ("Sensor já registrado").
 - enviar_dados(id_sensor, valor, timestamp)
 - **Descrição:** Armazena dados recebidos de um sensor.
 - **Retorno:** Confirmação ou erro ("Sensor não encontrado").
 - consultar_dados(id_sensor, data_inicio, data_fim)
 - **Descrição:** Retorna dados históricos de um sensor.
 - **Retorno:** Lista de registros (valor, timestamp).
 - alertas_atuais()
 - **Descrição:** Lista sensores com valores fora do esperado.
 - **Retorno:** Lista de sensores com descrição do alerta.
-

9. Plataforma de Reservas de Restaurante

API do Servidor

- reservar_mesa(usuario, restaurante, data, horario, qtd_pessoas)
 - **Descrição:** Verifica disponibilidade e cria uma reserva.
 - **Retorno:** Sucesso ou erro ("Mesa indisponível" ou "Restaurante fechado").
- cancelar_reserva(usuario, id_reserva)
 - **Descrição:** Cancela uma reserva ativa.
 - **Retorno:** Sucesso ou erro ("Reserva já expirada").
- listar_restaurantes(cidade)
 - **Descrição:** Retorna restaurantes cadastrados em uma cidade.

- **Retorno:** Lista com nome, endereço e horário de funcionamento.
 - avaliar_restaurante(usuario, restaurante, nota, comentario)
 - **Descrição:** Registra uma avaliação para o restaurante.
 - **Retorno:** Confirmação ou erro ("Usuário não visitou o restaurante").
-

10. Sistema de Chamados de Suporte Técnico

API do Servidor

- abrir_chamado(usuario, titulo, descricao, prioridade)
 - **Descrição:** Cria um novo chamado no sistema.
 - **Retorno:** ID do chamado ou erro ("Prioridade inválida").
 - atribuir_chamado(id_chamado, tecnico)
 - **Descrição:** Atribui um chamado a um técnico.
 - **Retorno:** Sucesso ou erro ("Chamado já atribuído").
 - atualizar_status(id_chamado, novo_status)
 - **Descrição:** Altera o status do chamado (ex: "Em andamento", "Resolvido").
 - **Retorno:** Confirmação ou erro ("Status inválido").
 - consultar_chamados(usuario, filtro)
 - **Descrição:** Lista chamados do usuário (abertos, todos, etc.).
 - **Retorno:** Lista com ID, título, status e técnico responsável.
-

Observações:

- O grupo deve possuir no mínimo 2 e no máximo 3 integrantes.
- O grupo pode escolher qualquer uma das aplicações sugeridas ou propor outra similar.
- A implementação deve seguir o paradigma cliente/servidor com RPC.
- A aplicação cliente pode ser uma interface (texto ou gráfica) que permita executar cada um dos tipos de funções disponíveis na API
- A documentação do projeto deve incluir a descrição da API, exemplos de uso e justificativas das escolhas de implementação.
- O grupo deverá preparar uma apresentação com duração máxima de 15 minutos, na qual serão apresentadas as ferramentas e tecnologias utilizadas no desenvolvimento do trabalho, bem como uma breve demonstração da aplicação.